Functions in C

A function in C is a block of organized reusuable code that is performs a single related action. Every C program has at least one function, which is **main()**, and all the most trivial programs can define additional functions.

When the algorithm of a certain problem involves long and complex logic, it is broken into smaller, independent and reusable blocks. These small blocks of code are known by different names in different programming languages such as a **module**, a **subroutine**, a **function** or a **method**.

You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but logically the division is such that each function performs a specific task.

A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

The C standard library provides numerous built-in functions that your program can call. For example, **strcat()** to concatenate two strings, **memcpy()** to copy one memory location to another location, and many more functions.

Modular Programming in C

Functions are designed to perform a specific task that is a part of an entire process. This approach towards software development is called **modular programming**.

Modular programming takes a top-down approach towards software development. The programming solution has a main routine through which smaller independent modules (functions) are called upon.

Each function is a separate, complete and reusable software component. When called, a function performs a specified task and returns the control back to the calling routine, optionally along with result of its process.

The main advantage of this approach is that the code becomes easy to follow, develop and maintain.

Library Functions in C

C offers a number of **library functions** included in different **header files**. For example, the **stdio.h** header file includes printf() and scanf() functions. Similarly, the **math.h** header file includes a number of functions such as sin(), pow(), sqrt() and more.

These functions perform a predefined task and can be called upon in any program as per requirement. However, if you don't find a suitable library function to serve your purpose,

you can define one.

Defining a Function in C

In C, it is necessary to provide the forward declaration of the prototype of any function. The prototype of a library function is present in the corresponding header file.

For a user-defined function, its prototype is present in the current program. The definition of a function and its prototype declaration should match.

After all the statements in a function are executed, the flow of the program returns to the calling environment. The function may return some data along with the flow control.

Function Declarations in C

A **function declaration** tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately.

A function declaration has the following parts −

```
return_type function_name(parameter list);
```

For the above defined function max(), the function declaration is as follows −

```
int max(int num1, int num2);
```

Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration −

```
int max(int, int);
```

A function declaration is required when you define a function in one source file and you call that function in another file. In such cases, you should declare the function at the top of the file calling the function.

Parts of a Function in C

The general form of a function definition in C programming language is as follows −

```
return_type function_name(parameter list){

   body of the function
}
```

A function definition in C programming consists of a function header and a function body. Here are all the parts of a function −

- **Return Type** − A function may return a value. The return_type is the data type of the value the function returns. Some functions perform the desired operations without returning a value. In this case, the return_type is the keyword void.
- **Function Name** − This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Argument List** − An argument (also called parameter) is like a placeholder. When a function is invoked, you pass a value as a parameter. This value is referred to as the actual parameter or argument. The parameter list refers to the type, order, and number of the parameters of a function. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** − The function body contains a collection of statements that defines what the function does.

A function in C should have a return type. The type of the variable returned by the function must be the return type of the function. In the above figure, the add() function returns an int type.

Example: User-defined Function in C

In this program, we have used a user-defined function called **max()**. This function takes two parameters **num1** and **num2** and returns the maximum value between the two −

</> Open Compiler

```
#include <stdio.h>

/* function returning the max between two numbers */
int max(int num1, int num2){

   /* local variable declaration */
   int result;
```

```
   if(num1 > num2)
      result = num1;
   else
      result = num2;

   return result;
}

int main(){
   printf("Comparing two numbers using max() function: \n");
   printf("Which of the two, 75 or 57, is greater than the other? \n");
   printf("The answer is: %d", max(75, 57));

   return 0;
}
```

Output

When you runt this code, it will produce the following output –

```
Comparing two numbers using max() function:
Which of the two, 75 or 57, is greater than the other?
The answer is: 75
```

Calling a Function in C

While creating a C function, you give a definition of what the function has to do. To use a function, you will have to call that function to perform the defined task.

To call a function properly, you need to comply with the declaration of the function prototype. If the function is defined to receive a set of arguments, the same number and type of arguments must be passed.

When a function is defined with arguments, the arguments in front of the function name are called **formal arguments**. When a function is called, the arguments passed to it are the **actual arguments**.

When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

Example: Calling a Function

To call a function, you simply need to pass the required parameters along with the function name. If the function returns a value, then you can store the returned value. Take a look at the following example –

</> Open Compiler

```
#include <stdio.h>

/* function declaration */
int max(int num1, int num2);

int main (){

   /* local variable definition */
   int a = 100;
   int b = 200;
   int ret;

   /* calling a function to get max value */
   ret = max(a, b);

   printf( "Max value is : %d\n", ret );

   return 0;
}

/* function returning the max between two numbers */
int max(int num1, int num2){

   /* local variable declaration */
   int result;

   if (num1 > num2)
      result = num1;
   else
      result = num2;

   return result;
}
```

Output

We have kept max() along with main() and compiled the source code. While running the final executable, it would produce the following result –

```
Max value is : 200
```

The main() Function in C

A C program is a collection of one or more functions, but one of the functions must be named as **main()**, which is the entry point of the execution of the program.

From inside the main() function, other functions are called. The main() function can call a library function such as printf(), whose prototype is fetched from the header file (stdio.h) or any other user-defined function present in the code.

In C, the order of definition of the program is not material. In a program, wherever there is main(), it is the entry point irrespective of whether it is the first function or not.

Note: In C, any function can call any other function, any number of times. A function can call itself too. Such a self-calling function is called a **recursive function**.

Function Arguments

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.

Formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways in which arguments can be passed to a function –

Sr.No	Call Type & Description
1	**Call by value** This method copies the actual value of an argument into the formal parameter of the function. In this case, changes made to the parameter inside the function have no effect on the argument.
2	**Call by reference** This method copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call. This means that changes made to the parameter affect the argument.

By default, C uses **call by value** to pass arguments. In general, it means the code within a function cannot alter the arguments used to call the function.